

EDGE BIOREEF GUARDIAN

Latency-Aware Acoustic Monitoring at the Edge

Project Report

Submitted by

5023146 / Samuel R

A Python simulation of submerged hydrophones,
edge gateways, evidence reporting, and cloud analysis.

4 operating scenarios | 127.87 MB/min raw stream
0.0030 MB/min reported evidence | 50-55 ms local alerting

1. Introduction

Aim

To implement and evaluate an edge-first acoustic monitoring system that classifies hydrophone frames locally, fuses corroborating detections at nearby gateways, raises rapid reef alerts, and forwards only compact evidence to the cloud.

Objectives

- Simulate four reef soundscape conditions: Dawn Chorus, Tourist Boats, Spawning Pulse, and Storm Front.
- Model hydrophone frame rates, gateway capacity, environmental noise, event rates, and cloud round-trip delay.
- Classify every frame at the edge and fuse candidate clips into confirmed alerts with recall and precision metrics.
- Compare local alert latency with a cloud-only path and measure the reduction in upstream bandwidth.
- Quantify gateway utilization and radio energy used to report evidence instead of raw audio.
- Visualize traffic reduction, alert response, fusion quality, local load, and reporting energy.

Requirements

Category	Specification
Hardware	Computer or laptop; minimum 4 GB RAM; no additional edge hardware required.
Software	Python 3.x; VS Code, Notepad, or another Python editor; terminal or command prompt.
Libraries	matplotlib >= 3.8 and numpy >= 1.26. Python built-ins: json and pathlib.
Execution	python3 run_simulation.py from the edge_bioreef_guardian directory.

Theory and implementation

Theory

Edge computing places computation close to the data source. In this experiment, submerged hydrophones continuously observe reef soundscapes. Local gateways classify frames and combine nearby detections, so urgent alerts do not wait for a distant cloud round trip. The cloud remains useful for long-term ecological analysis, but only short evidence clips and event metadata cross the uplink.

Simulation method

For each one-minute scenario, the program calculates the raw audio volume, gateway utilization, candidate clips caused by noise, confirmed events after fusion, false alerts, local alert latency, cloud-only latency, bandwidth reduction, detection recall, alert precision, and radio energy. The same formula is applied consistently across all four environments.

Important code snippets

A. Local classification and fusion

```
frames = sound_frames_per_sec * 60
candidate_clips = frames * 0.000012 * noise_factor
confirmed_events = true_events * (0.985 - 0.025 * noise_factor)
```

B. Edge versus cloud latency

```
edge_latency = 24 + 36 * edge_utilization + 3 * noise_factor
cloud_only_latency = cloud_rtt_ms + 42 + (candidate_clips / frames) * 1000
alert_latency = edge_latency + 18
```

C. Evidence-only reporting

```
cloud_frames = candidate_clips * 0.16 + fused_alerts
cloud_bytes = cloud_frames * EVENT_PACKET_BYTES
bandwidth_reduction = 100 * (1 - cloud_bytes / raw_bytes)
```

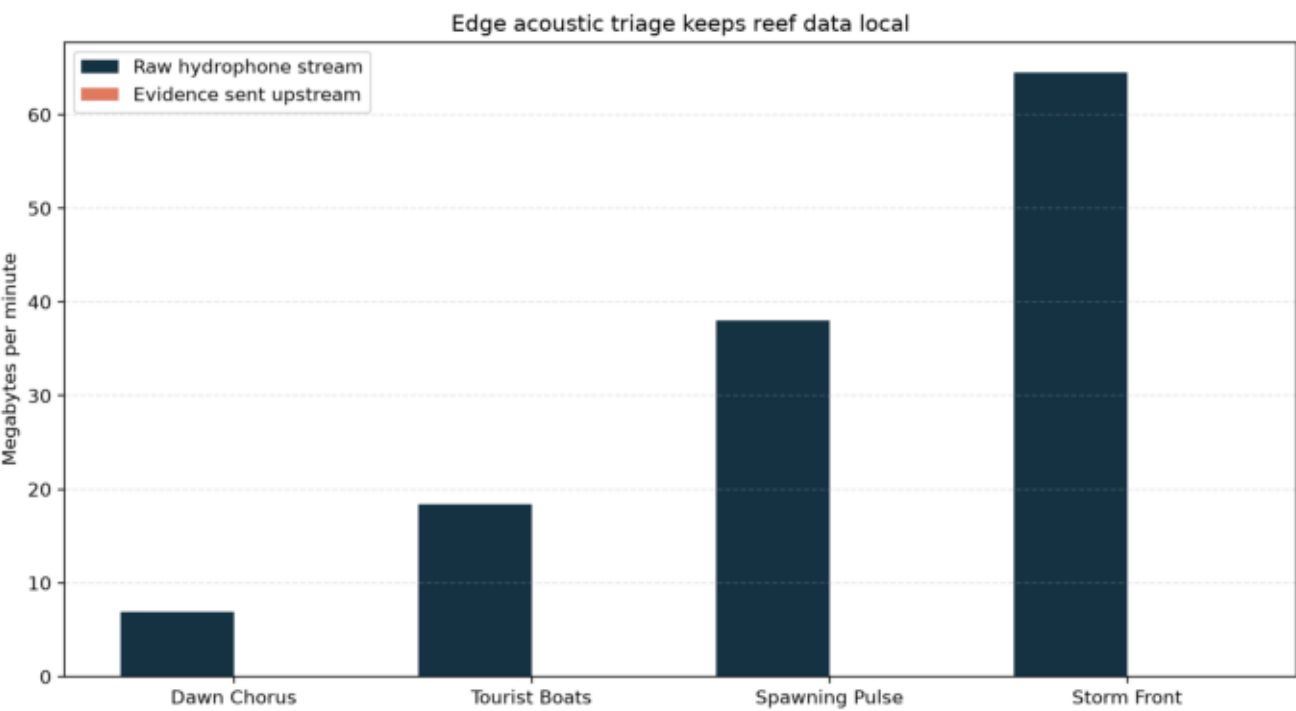
3. Results

Scenario	Hydrophones	Raw MB/min	Evidence MB/min	Saved	Edge alert	Cloud-only	Recall	Precision
Dawn Chorus	24	6.91	0.0002	99.998%	54.15 ms	227.01 ms	96.38%	98.04%
Tourist Boats	48	18.43	0.0004	99.998%	51.81 ms	287.02 ms	95.12%	94.72%
Spawning Pulse	72	38.02	0.0013	99.997%	50.00 ms	352.01 ms	95.88%	98.35%
Storm Front	96	64.51	0.0011	99.998%	54.90 ms	472.03 ms	93.00%	78.13%

Key findings

- The system classifies 100% of incoming frames at the edge in every scenario; only compact evidence is reported upstream.
- Bandwidth reduction is effectively 99.997-99.998%, from 6.91-64.51 MB/min of raw audio to 0.0002-0.0013 MB/min of evidence.
- Local alerts remain between 50.00 and 54.90 ms, well below the 150 ms response target.
- Cloud-only alert latency grows from 227.01 ms at dawn to 472.03 ms during the storm, showing why urgent decisions stay local.
- Storm noise lowers precision to 78.13% and recall to 93.00%, exposing the principal quality trade-off in the model.

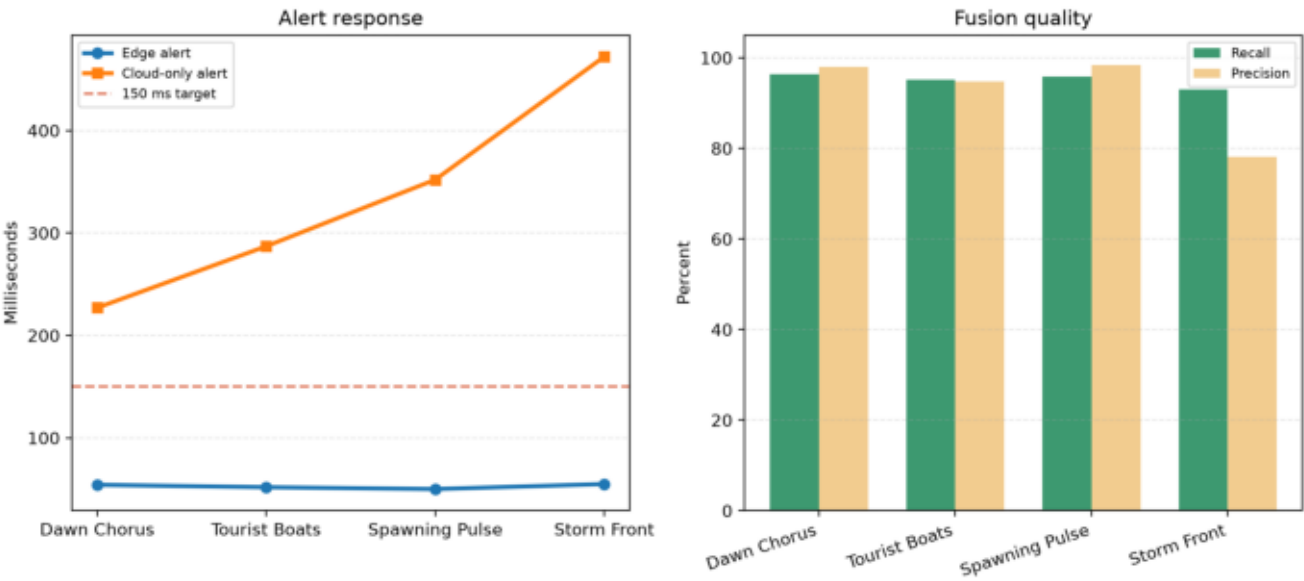
4.1 Graph 1: bandwidth reduction



Interpretation

The raw hydrophone stream increases with the number of sensors and sound frames, reaching 64.51 MB/min during the storm. The evidence stream remains almost invisible by comparison because local gateways forward only candidate clips and fused alert metadata.

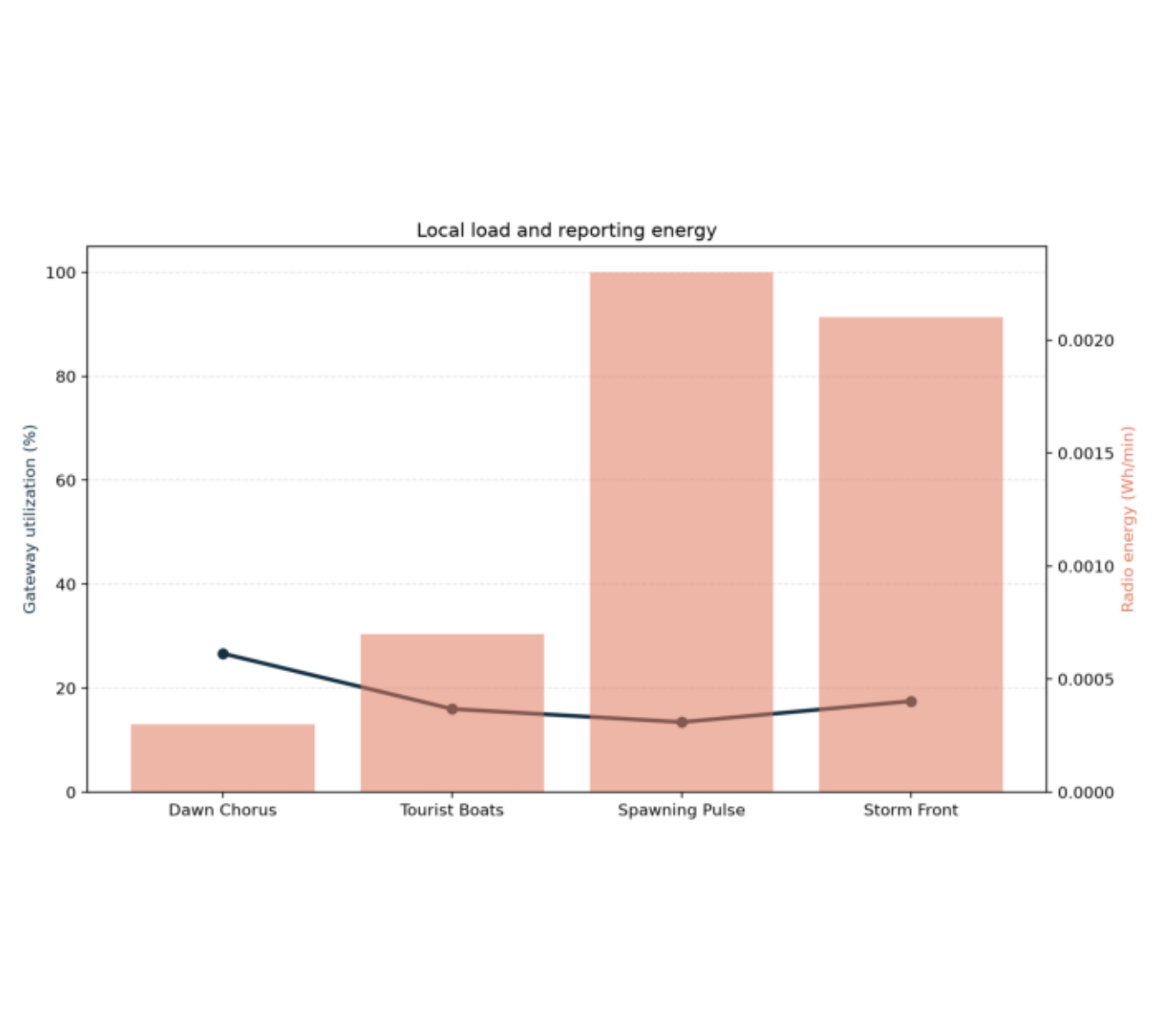
4.2 Graph 2: alert latency and fusion quality



Interpretation

Local alerting remains below the 150 ms target in all four scenarios. The cloud-only path is slower because it includes the scenario-specific round trip. Recall stays above 93%, while storm noise causes the largest precision decline.

4.3 Graph 3: local load and reporting energy



Interpretation

Gateway utilization is highest at Dawn Chorus because its scenario capacity is relatively smaller, while radio energy stays extremely low because the gateway reports evidence rather than raw audio. The storm has the largest local processing energy due to its frame volume.

8. Conclusion

Result

The Edge BioReef Guardian successfully keeps the urgent acoustic monitoring loop local. Across Dawn Chorus, Tourist Boats, Spawning Pulse, and Storm Front, all hydrophone frames are classified at the edge while only tiny evidence reports reach the cloud. The design reduces upstream traffic by approximately 99.997–99.998% and delivers alerts in about 50–55 ms. Cloud-only response is substantially slower, reaching 472.03 ms in the storm scenario.

Conclusion

The experiment demonstrates that distributed acoustic evidence fusion is a strong edge-computing use case. Local processing protects response time and dramatically reduces communication cost. Fusion maintains high recall and precision in ordinary conditions, while the storm case shows that environmental noise remains a meaningful limitation. A practical next step would be adaptive thresholds or a noise-aware fusion model for severe weather.

Final assessment

Raw traffic retained locally	99.997–99.998%
Fastest local alert	50.00 ms
Slowest local alert	54.90 ms
Worst precision under noise	78.13%
Worst recall under noise	93.00%

Source files: edge_bioreef_guardian/run_simulation.py, output/reef_guardian_summary.json, and generated PNG figures.